Practical:13 **Date:05-11-2026**

Advanced K-means clustering

Procedure:

Step 1: Data Ingestion & Combination

- Load both raw CSV files (1.csv and 2.csv) from your local directory into memory.
- Concatenate both datasets into a single unified dataset covering the full time period.

Step 2: Column Standardization & Data Cleaning

- Strip extra spaces from column names and dynamically map varying column names (e.g., matching customer ID, invoice number, unit price, date, and quantity to standard field names).
- Filter out records missing a Customer ID.
- Cast data types strictly (e.g., convert Customer IDs to integers and Invoice numbers to text).
- Filter out canceled transactions (invoices starting with 'C') and remove invalid records with non-positive quantities or prices.
- Parse invoice dates flexibly to handle mixed date string formats across both files.

Step 3: RFM & Customer Feature Engineering

- Calculate total monetary spend per row (Quantity * Price).
- Set an analysis cutoff date (one day past the latest dataset invoice date).
- Aggregate data at the customer level to construct 6 core behavioral features:
    - **Recency:** Days since the customer's last purchase.
    - **Frequency:** Total number of distinct invoices per customer.
    - **Monetary Value:** Total spend per customer.
    - **Average Basket Size:** Average item quantity purchased per invoice.
    - **Distinct Products:** Count of unique product codes purchased.
    - **Average Unit Price:** Average unit price of items purchased.

Step 4: Outlier Capping & Data Transformation

- Cap extreme upper-end values (99.5th percentile) on skewed features like Monetary and Average Basket Size to prevent extreme outliers from distorting cluster centroids.
- Apply a Log Transformation (log1p) across features to unskew skewed distributions.
- Standardize features using Z-score scaling (StandardScaler) so every feature contributes equally to distance calculations.

Step 5: Cluster Evaluation (Elbow & Silhouette Analysis)

- Iterate K-Means across a range of cluster values ($k = 2$ through 15).
- Calculate WCSS (Within-Cluster Sum of Squares / Inertia) to plot the Elbow Curve.
- Calculate Silhouette Scores to measure cluster separation and cohesion.
- Plot Elbow and Silhouette curves side-by-side to visually inspect optimal cluster boundaries.

Step 6: Hyperparameter Grid Search Optimization

- Run a grid search across candidate values ($k \in [3, 4, 5, 6]$), initialization strategies (k-means++ vs. random), and initialization runs (n_init).

- Evaluate each parameter combination using the Silhouette Score and select the highest-scoring parameter set.

Step 7: Final Model Fitting & Customer Segmentation

- Train the final K-Means model using the optimal hyperparameters.
- Predict and append the resulting cluster labels back to each customer's profile.

Step 8: Visualization & Cluster Profiling

- **2D PCA Projection:** Reduce the feature space to 2 Principal Components using PCA and produce a scatter plot showing cluster separation alongside cluster centroids.
- **Cluster Distribution:** Plot a bar chart showing the headcount/size distribution of customers across each identified segment.
- **Radar / Spider Chart:** Normalize feature means per cluster to display radar profiles, making it easy to compare and interpret customer segments (e.g., high-value loyalists vs. recent low-spend buyers).

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from math import pi
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from sklearn.model_selection import ParameterGrid
# 1. DATA LOADING
path1 = r"C:\Users\Lenovo\Downloads\1.csv"
path2 = r"C:\Users\Lenovo\Downloads\2.csv"
print("Loading CSV files...")
df1 = pd.read_csv(path1, encoding='ISO-8859-1')
df2 = pd.read_csv(path2, encoding='ISO-8859-1')
```

```
# Combine both files into a single full dataset
df = pd.concat([df1, df2], ignore_index=True)
print("=== First 10 Rows ===")
print(df.head(10))
print("\n=== Data Info & Missing Values ===")
print(df.info())
print("\nMissing Values Count:")
print(df.isnull().sum())
```

```
# 2. DYNAMIC COLUMN MAPPING & CLEANING
df.columns = df.columns.str.strip()
col_map = {}
for col in df.columns:
    c_lower = col.lower()
    if 'customer' in c_lower:
        col_map[col] = 'CustomerID'
    elif 'invoice' in c_lower and 'date' not in c_lower:
        col_map[col] = 'Invoice'
    elif 'date' in c_lower:
        col_map[col] = 'InvoiceDate'
    elif 'quantity' in c_lower:
        col_map[col] = 'Quantity'
    elif 'price' in c_lower:
        col_map[col] = 'Price'
    elif 'stock' in c_lower or 'code' in c_lower:
        col_map[col] = 'StockCode'
```

```
df.rename(columns=col_map, inplace=True)
df_clean = df.dropna(subset=['CustomerID']).copy()
df_clean['CustomerID'] = df_clean['CustomerID'].astype(int)
df_clean['Invoice'] = df_clean['Invoice'].astype(str)
df_clean = df_clean[~df_clean['Invoice'].str.startswith('C')]
df_clean = df_clean[(df_clean['Quantity'] > 0) & (df_clean['Price'] > 0)]
# Updated with format='mixed' to handle slash/dash date string formats
df_clean['InvoiceDate'] = pd.to_datetime(df_clean['InvoiceDate'], format='mixed')
print(f"\nCleaned Dataset Row Count: {len(df_clean)}")
print(f"Unique Customers (Cleaned): {df_clean['CustomerID'].nunique()}")
# 3. FEATURE ENGINEERING
df_clean['TotalSpend'] = df_clean['Quantity'] * df_clean['Price']
analysis_date = df_clean['InvoiceDate'].max() + pd.Timedelta(days=1)
features_df = df_clean.groupby('CustomerID').agg(
    Recency=('InvoiceDate', lambda x: (analysis_date - x.max()).days),
    Frequency=('Invoice', 'nunique'),
    Monetary=('TotalSpend', 'sum'),
    AvgBasketSize=('Quantity', lambda x: x.sum() / df_clean.loc[x.index, 'Invoice'].nunique()),
    NumDistinctProducts=('StockCode', 'nunique'),

    AvgUnitPrice=('Price', 'mean')
).reset_index()
print("\n=== Aggregated Customer Features (First 5) ===")
print(features_df.head())
for col in ['Monetary', 'AvgBasketSize']:
    cap_val = features_df[col].quantile(0.995)
    features_df[col] = np.where(features_df[col] > cap_val, cap_val, features_df[col])
feature_cols = ['Recency', 'Frequency', 'Monetary', 'AvgBasketSize', 'NumDistinctProducts', 'AvgUnitPrice']
df_log = np.log1p(features_df[feature_cols])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_log)
# 4. K-MEANS ELBOW & SILHOUETTE ANALYSIS
wcss = []
silhouette_scores = []
k_range = range(2, 16)
```

```
k_range = range(2, 16)
for k in k_range:
    kmeans_temp = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_temp.fit(X_scaled)
    wcss.append(kmeans_temp.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled, kmeans_temp.labels_))
fig, ax = plt.subplots(1, 2, figsize=(15, 5))
ax[0].plot(k_range, wcss, marker='o', color='b')
ax[0].set_title('Elbow Method (WCSS vs K)')
ax[0].set_xlabel('Number of Clusters (k)')
ax[0].set_ylabel('WCSS / Inertia')
ax[0].grid(True)
ax[1].plot(k_range, silhouette_scores, marker='s', color='g')
ax[1].set_title('Silhouette Score vs K')
ax[1].set_xlabel('Number of Clusters (k)')
ax[1].set_ylabel('Silhouette Score')
ax[1].grid(True)
plt.tight_layout()
plt.show()

# 5. HYPERPARAMETER OPTIMIZATION
param_grid = {
    'n_clusters': [3, 4, 5, 6],
    'init': ['k-means++', 'random'],
    'n_init': [10, 20, 50]
}
best_score = -1
best_params = None
for params in ParameterGrid(param_grid):
    km = KMeans(n_clusters=params['n_clusters'],
                init=params['init'],
                n_init=params['n_init'],
                random_state=42)
    labels = km.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    if score > best_score:
        best_score = score
        best_params = params
```

```
print("\n=== Hyperparameter Optimization Results ===")
print(f"Best Parameters: {best_params}")
print(f"Best Silhouette Score: {best_score:.4f}")
# 6. FINAL CLUSTERING & VISUALIZATIONS
final_kmeans = KMeans(**best_params, random_state=42)
features_df['Cluster'] = final_kmeans.fit_predict(X_scaled)
pca = PCA(n_components=2)
pca_components = pca.fit_transform(X_scaled)
centroids_pca = pca.transform(final_kmeans.cluster_centers_)
plt.figure(figsize=(9, 6))
sns.scatterplot(x=pca_components[:, 0], y=pca_components[:, 1], hue=features_df['Cluster'], palette='Set1', alpha=0.6)
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], s=250, c='black', marker='X', label='Centroids')
plt.title('PCA Cluster Scatter Plot (First 2 Components)')
plt.xlabel(f'PCA 1 ({pca.explained_variance_ratio_[0]*100:.1f}% Variance)')
plt.ylabel(f'PCA 2 ({pca.explained_variance_ratio_[1]*100:.1f}% Variance)')
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(7, 4))
sns.countplot(x='Cluster', data=features_df, palette='Set1')
```

```
plt.title('Cluster Distribution / Sizes')
plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.grid(axis='y')
plt.show()
# RADAR CHART
cluster_summary = pd.DataFrame(X_scaled, columns=feature_cols)
cluster_summary['Cluster'] = features_df['Cluster']
radar_df = cluster_summary.groupby('Cluster').mean().reset_index()
categories = list(radar_df)[1:]
N = len(categories)
angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1]
plt.figure(figsize=(8, 8))
ax = plt.subplot(111, polar=True)

for i, row in radar_df.iterrows():
    values = row[categories].tolist()
    values += values[:1]
    ax.plot(angles, values, linewidth=2, linestyle='solid', label=f'Cluster {int(row["Cluster"])}')
    ax.fill(angles, values, alpha=0.1)
plt.xticks(angles[:-1], categories)
ax.set_rlabel_position(0)
plt.title('Radar Chart: Scaled Mean Profiles per Cluster', y=1.08)
plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
plt.show()
```

```
=== First 10 Rows ===
  InvoiceNo StockCode                          Description  Quantity       InvoiceDate  UnitPrice  CustomerID         Country
0    489434     85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS        12  12-01-2009 07:45       6.95     13085.0  United Kingdom
1    489434    79323P                   PINK CHERRY LIGHTS        12  12-01-2009 07:45       6.75     13085.0  United Kingdom
2    489434    79323W                  WHITE CHERRY LIGHTS        12  12-01-2009 07:45       6.75     13085.0  United Kingdom
3    489434     22041         RECORD FRAME 7" SINGLE SIZE         48  12-01-2009 07:45       2.10     13085.0  United Kingdom
4    489434     21232       STRAWBERRY CERAMIC TRINKET BOX        24  12-01-2009 07:45       1.25     13085.0  United Kingdom
5    489434     22064            PINK DOUGHNUT TRINKET POT        24  12-01-2009 07:45       1.65     13085.0  United Kingdom
6    489434     21871                    SAVE THE PLANET MUG        24  12-01-2009 07:45       1.25     13085.0  United Kingdom
7    489434     21523   FANCY FONT HOME SWEET HOME DOORMAT        10  12-01-2009 07:45       5.95     13085.0  United Kingdom
8    489435     22350                      CAT BOWL          12  12-01-2009 07:46       2.55     13085.0  United Kingdom
9    489435     22349       DOG BOWL , CHASING BALL DESIGN        12  12-01-2009 07:46       3.75     13085.0  United Kingdom

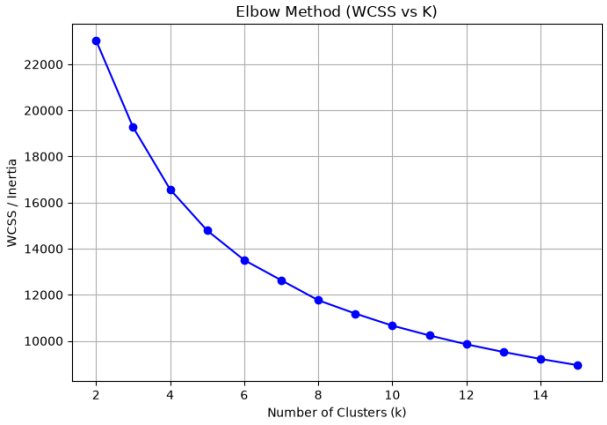
=== Data Info & Missing Values ===
<class 'pandas.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
 #   Column       Non-Null Count    Dtype
---  ------       --------------    -----
 0   InvoiceNo    1067371 non-null  str
 1   StockCode    1067371 non-null  str
 2   Description  1062989 non-null  str
 3   Quantity     1067371 non-null  int64
 4   InvoiceDate  1067371 non-null  str
 5   UnitPrice    1067371 non-null  float64
 6   CustomerID   824364 non-null   float64
 7   Country      1067371 non-null  str
dtypes: float64(2), int64(1), str(5)
memory usage: 65.1 MB
None
```

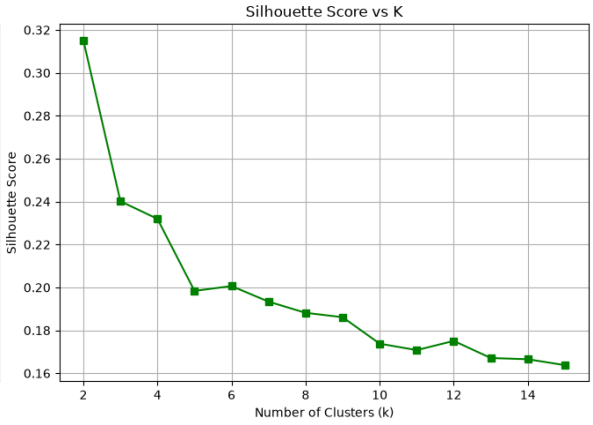
```
Missing Values Count:
InvoiceNo          0
StockCode          0
Description     4382
Quantity           0
InvoiceDate        0
UnitPrice          0
CustomerID    243007
Country            0
dtype: int64

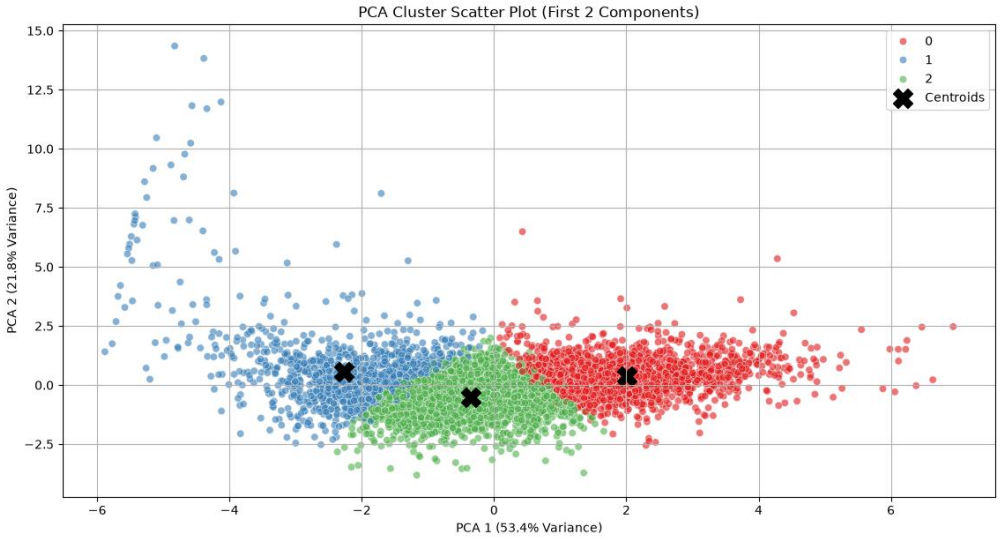
Cleaned Dataset Row Count: 805549
Unique Customers (Cleaned): 5878

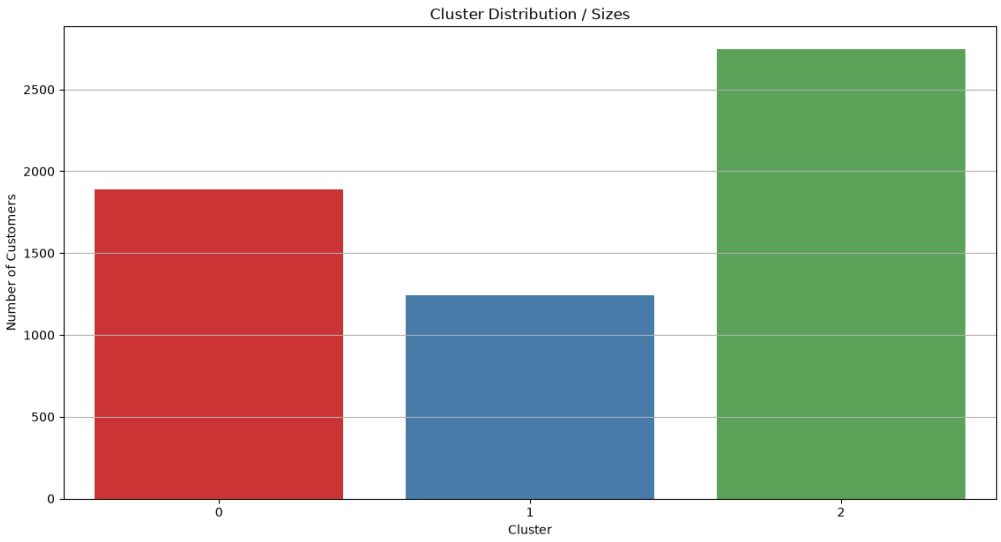
=== Aggregated Customer Features (First 5) ===
   CustomerID  Recency  Frequency  Monetary  AvgBasketSize  NumDistinctProducts  AvgUnitPrice
0       12346      326         12  77556.46    6190.416667                   27      6.100000
1       12347        2          8   5633.32     410.750000                  126      2.546087
2       12348       75          5   2019.40     542.800000                   25      3.786275
3       12349       19          4   4428.69     406.000000                  138      8.459657
4       12350      310          1    334.40     197.000000                   17      3.841176

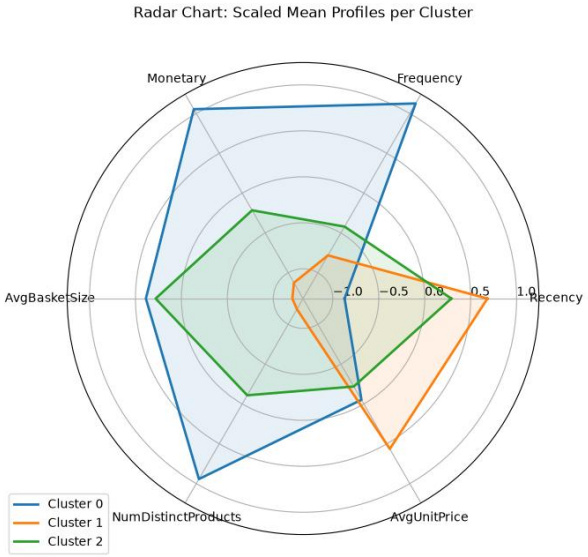
=== Hyperparameter Optimization Results ===
Best Parameters: {'init': 'random', 'n_clusters': 3, 'n_init': 10}
Best Silhouette Score: 0.2405
```








Cluster Distribution / Sizes
Number of Customers
2500
2000
1500
1000
500
0
0
1
2
Cluster


Radar Chart: Scaled Mean Profiles per Cluster
Monetary
Frequency
AvgBasketSize
Recency
−1.0
−0.5
0.0
0.5
1.0
NumDistinctProducts
AvgUnitPrice
Cluster 0
Cluster 1
Cluster 2